

Dokumentacja końcowa

Projekt: Wizualizacja grafu planarnego (część w języku Java)

1. Wprowadzenie

Celem projektu jest stworzenie aplikacji w języku Java, której zadaniem jest wizualizacja grafu na podstawie współrzędnych wyznaczonych przez część w C.

2. Zakres funkcjonalny systemu

Aplikacja pozwala na sterowanie programem za pomocą interfejsu. Po wciśnięciu klawiszy wczytaj graf i wczytaj współrzędne, a następnie wybranie odpowiednich plików na ekranie wyświetli się wizualizacja danego grafu.

3. Sposób uruchomienia programu

Program można uruchomić na dwa sposoby:

Program można bezpośrednio uruchomić z poziomu terminala. W tym celu należy skompilować plik komendą 'javac <nazwa pliku>', a następnie go uruchomić: 'java <nazwa pliku>'.

Innym sposobem jest użycie gotowego środowiska programistycznego, który ma wbudowany kompilator. W tym przypadku proces kompilacji i uruchomienia odbywa się automatycznie. Użytkownik musi tylko nacisnąć zieloną strzałkę.

4. Format danych wejściowych

Do uruchomienia programu potrzebne są dwa pliki wejściowe: jeden z informacjami o grafie, a drugi z jego współrzędnymi

Dane wejściowe należy zapisać w pliku tekstowym w podanych formatach:

1) <nazwa_krawędzi> <wierzchołek_A> <wierzchołek_B> <waga>

Przykład:

AB 1 2 1

BC 2 3 1

CD 3 4 1

DB 4 2 1.407

Wierzchołki muszą być liczbami całkowitymi. Waga musi być większa od zera.

Format tekstowy:

2) <wierzchołek> <x> <y>

Przykład:

4 19.285 -9.111

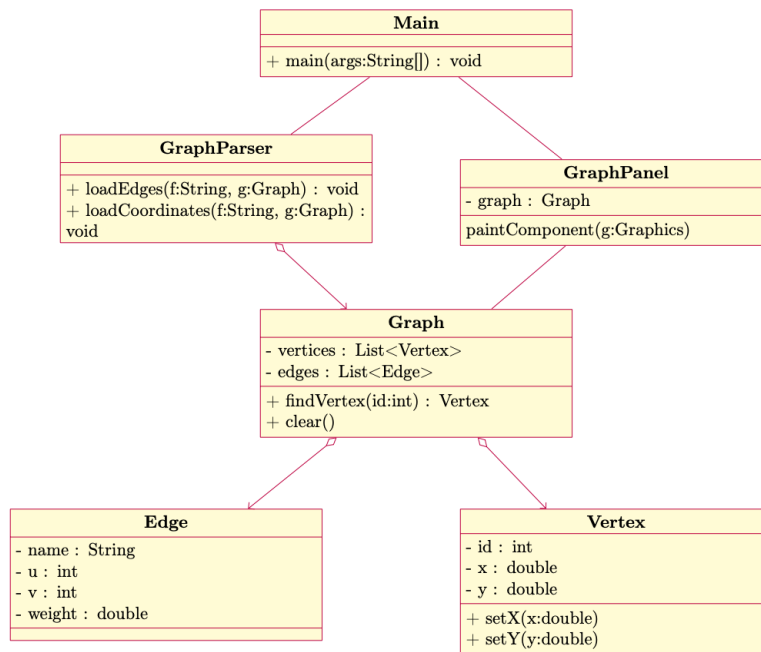
2 31.133 47.189

3 66.326 1.637

1 16.799 109.086

5. Struktura implementacyjna systemu

Cały program składa się z sześciu klas.



W klasach Vertex oraz Edge przechowywane są informacje o wierzchołku i krawędzi grafu.

```
public class Vertex { 11 usages
    private final int id; 2 usages

    private double x; 2 usages
    private double y; 2 usages

    public Vertex(int id) { 2 usages
        this.id = id;
    }

    public int getId() { 2 usages
        return id;
    }

    public double getX() { 3 usages
        return x;
    }

    public double getY() { 3 usages
        return y;
    }

    public void setX(double x) { this.x = x; }

    public void setY(double y) { this.y = y; }
}
```

```

public class Edge { 5 usages
    private final String name; 2 usages

    private final int u; 2 usages
    private final int v; 2 usages

    private final double weight; 2 usages

    public Edge(String name, int u, int v, double weight) { 1 usage
        this.name = name;
        this.u = u;
        this.v = v;
        this.weight = weight;
    }

    public String getName() { return name; }

    public int getU() { return u; }

    public int getV() { return v; }

    public double getWeight() { return weight; }
}

```

Klasa Graph posiada dwie listy: jedna z wierzchołkami, drugą z krawędziami. W tej klasie znajduje się metoda findVertex(), która przeszukuje listę, aby znaleźć wierzchołek po jego numerze id.

```

public Vertex findVertex(int id) { 5 usages
    for (Vertex v : vertices) {
        if (v.getId() == id)
            return v;
    }

    return null;
}

```

Klasa GraphParser posiada dwie metody, które służą do czytania danych z plików. Znajdują się w niej dwie metody loadEdges() oraz loadCoordinates().

```
public static void loadEdges(String filename, Graph graph) throws Exception { 1 usage

    graph.clear();

    BufferedReader br = new BufferedReader(new FileReader(filename));

    String line;

    while ((line = br.readLine()) != null) {

        String[] parts = line.trim().split(regex: "\\s+");

        if (parts.length < 4)
            continue;

        String name = parts[0];

        int u = Integer.parseInt(parts[1]);
        int v = Integer.parseInt(parts[2]);

        double weight = Double.parseDouble(parts[3]);

        graph.addEdge(new Edge(name, u, v, weight));

        if (graph.findVertex(u) == null)
            graph.addVertex(new Vertex(u));

        if (graph.findVertex(v) == null)
            graph.addVertex(new Vertex(v));
    }

    br.close();
}
```

```

public static void loadCoordinates(String filename, Graph graph) throws Exception { 1 usage

    BufferedReader br = new BufferedReader(new FileReader(filename));

    String line;

    while ((line = br.readLine()) != null) {

        String[] parts = line.trim().split(regex: "\\s+");

        if (parts.length < 3)
            continue;

        int id = Integer.parseInt(parts[0]);

        double x = Double.parseDouble(parts[1]);
        double y = Double.parseDouble(parts[2]);

        Vertex vertex = graph.findVertex(id);

        if (vertex != null) {

            // Skalowanie żeby było lepiej widać
            vertex.setX(x * 5);
            vertex.setY(y * 5);

        }

    }

    br.close();
}

```

Klasa GraphPanel odpowiada za wizualizację grafu. W tym celu została użyta biblioteka Swing.

```
// Rysowanie krawędzi
for (Edge edge : graph.getEdges()) {

    Vertex v1 = graph.findVertex(edge.getU());
    Vertex v2 = graph.findVertex(edge.getV());

    if (v1 == null || v2 == null)
        continue;

    int x1 = (int) v1.getX() + 100;
    int y1 = (int) v1.getY() + 100;

    int x2 = (int) v2.getX() + 100;
    int y2 = (int) v2.getY() + 100;

    g2.setColor(Color.BLACK);
    g2.drawLine(x1, y1, x2, y2);
}
}
```

```
// Rysowanie wierzchołków
for (Vertex vertex : graph.getVertices()) {

    int x = (int) vertex.getX() + 100;
    int y = (int) vertex.getY() + 100;

    g2.setColor(Color.CYAN);
    g2.fillOval(x: x - 10, y: y - 10, width: 20, height: 20);

    g2.setColor(Color.BLACK);
    g2.drawOval(x: x - 10, y: y - 10, width: 20, height: 20);

    g2.drawString(
        String.valueOf(vertex.getId()),
        x: x - 4,
        y: y + 4
    );
}
}
```

Klasa Main odpowiada za utworzenie okna, gdzie jest wyświetlany graf, interaktywnych przycisków oraz uruchomienie programu rysowania grafu.

```
SwingUtilities.invokeLater(() -> {  
  
    JFrame frame = new JFrame( title: "Wizualizacja grafu planarnego");  
  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    frame.setSize( width: 1000, height: 800);  
  
    GraphPanel panel = new GraphPanel();  
  
    JButton loadGraphButton = new JButton( text: "Wczytaj graf");  
    JButton loadCoordsButton = new JButton( text: "Wczytaj współrzędne");  
  
    JPanel topPanel = new JPanel();  
  
    topPanel.add(loadGraphButton);  
    topPanel.add(loadCoordsButton);  
}
```

```
// Wczytywanie grafu
loadGraphButton.addActionListener( ActionEvent e -> {

    JFileChooser chooser = new JFileChooser();

    int result = chooser.showOpenDialog(frame);

    if (result == JFileChooser.APPROVE_OPTION) {

        File file = chooser.getSelectedFile();

        try {

            GraphParser.loadEdges(
                file.getAbsolutePath(),
                panel.getGraph()
            );

            panel.repaint();

        } catch (Exception ex) {

            JOptionPane.showMessageDialog(frame, message: "Błąd wczytywania grafu");

        }

    }

});
```

```

// wczytywanie współrzędnych
loadCoordsButton.addActionListener( ActionEvent e -> {

    JFileChooser chooser = new JFileChooser();

    int result = chooser.showOpenDialog(frame);

    if (result == JFileChooser.APPROVE_OPTION) {

        File file = chooser.getSelectedFile();

        try {

            GraphParser.loadCoordinates(
                file.getAbsolutePath(),
                panel.getGraph()
            );

            panel.repaint();

        } catch (Exception ex) {

            JOptionPane.showMessageDialog(
                frame,
                message: "Błąd wczytywania współrzędnych"
            );

        }
    }
}

```

6. Komunikaty błędów

W przypadku podania nieprawidłowych danych program wyświetli komunikaty:

- przy podaniu złych plików wyświetli się komunikat: „Błąd wczytywania grafu/współrzędnych”

7. Testy

Dane wejściowe:

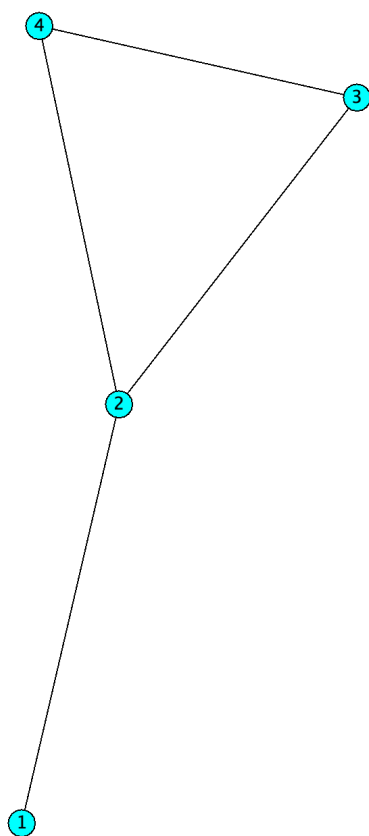
```
AB 1 2 1
BC 2 3 1
CD 3 4 1
DA 4 1 1.407
```

rys a graf

```
4 19.285 -9.111
2 31.133 47.189
3 66.326 1.637
1 16.799 109.086
```

rys b współrzędne

Wynik działania:



Dane wejściowe:

A	1	2	10
B	2	3	12
C	3	1	15

rys c graf

3	9.659	28.723
1	61.107	54.923
2	58.073	-2.732

rys d współrzędne

Wynik działania:

